

TECHNICAL DEVELOPMENT REPORT

AI-Powered Scholarship Matching System — Phase 2

On-Demand Matching | PDF Report Generator | Interest-Based Search

Development Date	System Version	Total Students	AI Model
6 July 2026	v2.0	50 SPM Students	Groq Llama-3.3-70b

1. Introduction

This report presents the technical enhancements implemented in Phase 2 of the AI-Powered Scholarship Matching System for SPM students at Sekolah Berasrama Penuh Ibnu Khaldun. Phase 2 focuses on three major additions: (1) on-demand per-student matching triggered at login, (2) a downloadable PDF report generator, and (3) an interest-based scholarship search function.

The overall system retains the established 3-agent pipeline architecture from Phase 1, with stronger inter-agent integration and an improved student-facing portal experience.

2. System Architecture (3-Agent Pipeline)

The system operates as an on-demand 3-agent pipeline triggered when a student logs in:

Agent	Role	Technology	Output
Agent 1	Student profiling — reads SPM, PAJSK, RIAS data from Excel and produces output.json	Node.js XLSX library	output.json (50 profiles)
Agent 2	Scholarship matching — computes academic, PAJSK, psychometric, aspiration and income scores	Python ScrapAPI	matches_{id}.json interest_{id}.json
Agent 3	PDF report generation — reads matching results and produces a printable scholarship report	Python ReportLab	report_{id}.pdf

All inter-agent communication is handled through the Node.js server (port 3333) using child_process.spawn to invoke Python scripts as background processes.

3. Feature 1 — On-Demand Matching

3.1 Problem Statement

In Phase 1, Agent 2 processed all 50 students simultaneously before any results could be displayed. This batch approach was slow and unsuitable for a real-time student portal that requires immediate feedback after login.

3.2 Solution

A new function `run_single(student_id)` was added to `agent2.py`. This function processes only the one student who has just logged in, reducing Groq API calls from approximately 500 (full batch) to approximately 10 (single student).

Technical Changes — `agent2.py`

- Added `run_single(student_id, progress_cb=None)` function
- Loads only the specific student's profile from `output.json`
- Runs `batch_aspiration` for 1 student across all scholarships
- Saves result to `Agent2/matches_{student_id}.json`
- CLI entry: `python agent2.py --single`

New Endpoints — `server.js`

Endpoint	Function	Method
<code>/students</code>	Returns student list (name, ID, password) for the login dropdown	GET
<code>/start-match</code>	Spawns Python <code>agent2.py --single <id></code> as a child process after login	POST
<code>/match-status</code>	Returns current matching progress (running / done / error)	GET
<code>/match-result</code>	Returns the JSON matching result once complete	GET

UI Enhancements — `index.html`

- Student name dropdown replaces manual ID input — data fetched from `/students`
- Password auto-fills when a student name is selected
- Groq API key stored in `.env` file at project root — never exposed to the student UI
- Loading spinner replaces the raw technical log box
- Dynamic status text: 'Evaluating X of Y scholarships...'
- Frontend polls `/match-status` every 1.5 seconds until status is done

BEFORE (Phase 1)	AFTER (Phase 2)
Runs all 50 students at once ~500 Groq API calls per session Long waiting time for every student	Runs only for the student who logged in ~10 Groq API calls per session Fast, personalised response

4. Agent 3 — PDF Report Generator

4.1 Purpose

Agent 3 serves as the presentation layer of the system. It reads raw matching data from Agent 2, formats it into a structured document, and generates a PDF file that students can download and print without needing to log in again.

4.2 Technical Implementation

Library: ReportLab (Python)

- Installed via: `pip install reportlab`
- Generates PDF directly in Python — no browser or LibreOffice dependency
- Full support for layouts, tables, colours, and fonts

PDF Report Contents

- Official header — Ministry of Education Malaysia
- Student information: Full Name, Student ID, Date Generated
- Top 5 Scholarships table: Name, Provider, Score /100, Eligibility Status
- Score factor breakdown: Academic, Aspiration (colour-coded chips)
- Official footer with page numbers

Agent 3 Data Flow

Step	Action	Detail	Output
1	Read Agent 2 data	Open Agent2/matches_{student_id}.json	Student top5 data
2	Build PDF	Generate document using ReportLab SimpleDocTemplate	PDF in memory
3	Save file	Write to Agent3/report_{student_id}.pdf	PDF file on disk
4	Serve to student	Endpoint /download-report streams the PDF file	Browser download

4.3 New Endpoints — server.js

- POST /start-report — spawns `agent3.py --single`
- GET /report-status — returns status (running / done / error)
- GET /download-report — streams the PDF file directly to the browser

4.4 User Experience

- PDF is generated automatically after matching completes — no extra action required
- Green 'Download PDF Report' button appears when the file is ready
- Student can save and print for future reference without logging in again

5. Feature 2 — Interest-Based Scholarship Search

5.1 Background

The original matching system derived a student's field of interest from IMK (Career Interest Instrument) records stored in the Excel dataset. However, a student's actual interest may differ from or be more specific than what is recorded in the IMK. This feature allows students to freely type their own interest to discover scholarships that are most relevant to their chosen field.

5.2 Scoring Algorithm Differences

Score Factor	Original Matching	Interest Search
Academic (GP Score)	35%	35%
Co-curriculum (PAJSK)	20%	20%
Psychometric (Holland Code)	15%	15%
Aspiration (field of interest)	15% (from IMK data)	30% (from student input)
Family income category	15%	0% (not applied)
TOTAL	100%	100%

The aspiration weight is raised to 30% to reflect the student's chosen interest, while the family income factor is excluded because the purpose of this mode is purely to evaluate field relevance.

5.3 Technical Implementation

New Function — agent2.py

- Added `run_interest(student_id, interest, progress_cb=None)`
- Overrides the student's `fieldOfInterest` and `dreamCareer` with the typed interest text
- Runs `batch_aspiration` using the new interest for all scholarships
- Recomputes scores with aspiration weight at 30%
- Saves result to `Agent2/interest_{student_id}.json`
- CLI: `python agent2.py --interest`

New Endpoints — server.js

- POST `/start-interest` — accepts `{id, interest}`, spawns `agent2.py --interest`
- GET `/interest-status?id=&interest;=` — returns search progress
- GET `/interest-result?id=&interest;=` — returns JSON search results
- State key: `id::interest` (supports different searches for the same student)

UI Components — index.html

- Text input with placeholder: 'e.g. engineering, medicine, architecture, IT...'
- 'Search Scholarships' button (blue) with Enter key support
- 7 suggested quick-select tags: Engineering, Medicine, Law, Computer Science, Accounting, Architecture, Teaching
- Loading spinner with dynamic text: 'Evaluating X of Y scholarships for engineering...'
- Results table: Rank, Scholarship Name, Score, Eligibility, Breakdown (Academic + Interest)
- Appears automatically below the original Top 5 table once matching is complete

6. Files Modified

File	Status	Key Changes
Agent2/agent2.py	Modified	Added run_single() and run_interest() functions
Agent3/agent3.py	New	PDF generation script using ReportLab
Agent1/server.js	Modified	Added 9 new endpoints + interestState + .env loader
Agent1/index.html	Modified	Student dropdown, spinner UI, interest search, download button
.env	New	Stores GROQ_API_KEY — never exposed to the student UI

7. Security Considerations

- Groq API key stored in .env at project root — not sent to the frontend under any circumstance
- Student passwords loaded from Excel on the server side only — not included in /students API response
- Matching state stored in server memory — students can only access their own results
- All user inputs validated on the server before spawning child processes

8. Conclusion

Phase 2 successfully delivers three significant enhancements that improve both the performance and usability of the system. On-demand matching reduces API consumption and student waiting time dramatically. The PDF report generator enables students to save and print their scholarship results for future reference without re-authentication. The interest-based search gives students the freedom to explore scholarships according to their actual aspirations, independent of their IMK records.

The system continues to operate under the consistent 3-agent architecture — Agent 1 (profiling), Agent 2 (matching), and Agent 3 (reporting) — communicating through JSON files and well-defined REST endpoints.